

MÓDULO 1: Fundamentos y Tooling

1.1 Instalación y Toolchain

Rust se instala con rustup, el gestor de toolchains (no instalas "Rust" directamente, instalas un gestor de versiones de Rust).

Instalación (Linux/macOS)

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Verificar instalación

```
rustc --version # el compilador
```

```
cargo --version # el gestor de paquetes/build system
```

```
rustup --version # el gestor de toolchains
```

1.1.b Configuración del Entorno Profesional: rust-analyzer, clippy y rustfmt

Un entorno de producción en Rust no se limita al compilador; se apoya en tres herramientas estándar integradas directamente en el toolchain:

1. **rust-analyzer (Language Server Protocol - LSP):** Proporciona inferencia de tipos en tiempo real, expansión de macros en el editor, autocompletado semántico y chequeos continuos mediante cargo check en segundo plano.
2. **rustfmt (Formateador Oficial):** Estandariza el estilo de código siguiendo las directrices oficiales del lenguaje.
3. **clippy (Linter Estático Oficial):** Aplica más de 700 reglas de optimización, seguridad, ergonomía y detección de patrones anti-idiomáticos.

Instalación de componentes esenciales mediante rustup

```
rustup component add clippy rustfmt
```

Verificación de formateo en CI (falla si el código no cumple las reglas de estilo)

```
cargo fmt -- --check
```

Ejecución de clippy en modo estricto (rechaza warnings en pipelines de CI/CD)

```
cargo clippy --all-targets --all-features -- -D warnings
```

1.1.c Meta-Habilidad: Lectura de Errores y rustc --explain

El compilador de Rust (rustc) está diseñado como un motor de diagnóstico. Un reporte de error consta de cuatro partes fundamentales:

```
error[E0384]: cannot assign twice to immutable variable `puerto`  <-- [1] error y
resumen

--> src/main.rs:4:5                                <-- [2] Ubicación exacta
(archivo:línea:columna)

|
2 |     let puerto = 8080;
|
|         ----- first assignment to `puerto`      <-- [3] Contexto y declaración
original
3 |     println!("Puerto: {}", puerto);
4 |     puerto = 9000;
|
|     ^^^^^^^^^^^^^^^ cannot assign twice to immutable variable  <-- [4] Span visual
|

help: consider making this binding mutable  <-- [5] Sugerencia de solución
aplicable

|
2 |     let mut puerto = 8080;
|
|     +++
```

Inspección del catálogo de errores (rustc --explain)

Cada código E0XXX tiene una entrada enciclopédica integrada en el compilador con ejemplos de código erróneo y su solución idiomática:

Consulta directa en terminal de la causa y solución técnica de un error

rustc --explain E0384

♦ Los tres binarios que debes entender:

Binario	Rol
rustc	El compilador puro. Casi nunca lo invocas directamente.
cargo	Build system + gestor de dependencias. Tu herramienta diaria.
rustup	Gestiona versiones de Rust (stable, beta, nightly) y targets de compilación.

1.2 Estructura de un Proyecto Cargo

```
cargo new mi_proyecto
cd mi_proyecto
```

Esto genera:

```
mi_proyecto/
├── Cargo.toml      # Manifiesto: metadatos + dependencias
├── Cargo.lock      # Versiones exactas resueltas (no tocar a mano)
└── src/
    └── main.rs     # Punto de entrada del binario
```

Cargo.toml mínimo:

```
[package]
name = "mi_proyecto"
version = "0.1.0"
edition = "2021"
[dependencies]
# aquí van los crates externos
```



Comandos esenciales que usarás constantemente:

```
cargo build          # compila (debug, sin optimizar, rápido de
                      # compilar)
cargo build --release # compila optimizado (lento de compilar,
                      # rápido en ejecución)
cargo run            # compila y ejecuta
cargo check          # solo verifica tipos/borrow checker, NO
                      # genera binario (muy rápido, úsalo mientras iteras)
cargo test           # corre tests
cargo doc --open     # genera y abre documentación
```



Hábito clave desde el día 1: usa cargo check mientras escribes código y cargo run solo cuando quieras ejecutar de verdad. cargo check es 3-5x más rápido porque se salta la generación de código máquina.

1.2.b.Sintaxis de Documentación Técnica (/// y //!)

Rust compila los comentarios de documentación directamente en HTML mediante cargo doc e integra tests ejecutables (Doc Tests):

```
//! # Herramienta de Auditoría de Red
//!
Este componente expone primitivas de bajo nivel para la orquestación
de sockets y el escaneo de puertos.

/// Procede con el setup y la verificación técnica de los parámetros
de enlace para un nodo backend.
///
/// # Parámetros
/// * `host`: Endpoint de destino (IP o FQDN).
/// * `puerto`: Punto de acceso TCP (rango válido: `1..=65535`).
///
/// # Ejemplos
/// ```
/// let online = true;
/// assert!(online);
/// ```
pub fn inicializar_servicio(host: &str, puerto: u16) {
    println!("Desplegando listener en {}:{}", host, puerto);
}
```

1.3 Variables, Inmutabilidad e Inferencia de Tipos

En Rust, **las variables son inmutables por defecto**. Esto es una decisión de diseño deliberada, no una limitación: el compilador te obliga a declarar explícitamente cuándo algo puede cambiar.

```
fn main() {
    let x = 5;
    println!("x vale: {}", x);
    // x = 6; //  ERROR DE COMPILACIÓN – descomenta para ver el
error
    let mut y = 5; // mut hace explícita la intención de mutar
    println!("y antes: {}", y);
    y = 6;
    println!("y después: {}", y);}
```

! Error típico del compilador #1

Si descomentas `x = 6;`, obtienes:

```
error[E0384]: cannot assign twice to immutable variable `x`
```

```
--> src/main.rs:4:5
2 |         let x = 5;
  |         - first assignment to `x`
3 |         println!("x vale: {}", x);
4 |         x = 6;
  |         ^^^^^ cannot assign twice to immutable variable
```

```
help: consider making this binding mutable
```

```
2 |         let mut x = 5;
  |         +++
```

Lectura del error: Rust no solo te dice qué está mal, te sugiere la solución exacta (help:). Esto es constante en Rust: el compilador es tu maestro, no tu enemigo. Acostúmbrate a leer los mensajes completos, no solo la primera línea.

🧠 Diagrama mental: ¿Dónde vive x?

Stack (rápido, tamaño conocido en compile-time, LIFO)

main() frame	
x: i32 = 5	
y: i32 = 6	

← vive aquí, se libera automáticamente al salir del scope (final de main)

Para tipos primitivos como `i32`, todo vive en el stack. Cuando lleguemos a `String` o `Vec`, verás que parte de los datos vive en el **Heap**, y el stack solo guarda un puntero + metadata. Esto es la base de todo el Módulo 2 (Ownership), así que grábate esta imagen.

1.3.b Constantes, Variables Estáticas, Type Aliases y Casting Numérico

const vs static

Propiedad	const (Constante)	static (Variable Estática)
Inlining en compilación	Sí (se sustituye su valor en cada punto de uso).	No (posee una dirección de memoria fija en el binario).
Dirección de memoria	No garantizada / no fija.	Única durante todo el ciclo de vida del programa ('static).
Mutabilidad	Totalmente inmutable.	Requiere <code>unsafe</code> para modificarse (<code>static mut</code>), desaconsejado.
Caso de uso	Magic numbers, límites fijos, configuraciones puras.	Tablas de consulta globales, buffers compartidos globales.

```
// Constantes (const): se evalúan e inlinean durante la compilación.
const TIMEOUT_SEGUNDOS: u64 = 30;

// Estáticas (static): garantizan una ubicación de memoria fija en
el binario.
static MENSAJE_BIENVENIDA: &str = "SISTEMA INICIALIZADO";

// Soporte para literales numéricos con guiones bajos (_) como
separadores de legibilidad.
const MAX_CONEXIONES: u32 = 1_000_000;
const BYTES_BUFFER: usize = 0xFF_AA_00_11;

// Type Aliases (type): permiten inyectar semántica de dominio sobre
tipos primitivos.
type SocketPort = u16;
type MetricaMs = u64;

fn registrar_latencia(puerto: SocketPort, latencia: MetricaMs) {
    println!("Puerto {}: {} ms", puerto, latencia);
}
```

Casting Numérico Explícito con as

Rust **no realiza coerción implícita de tipos numéricos**. No es posible asignar un u8 a un u64 ni operar enteros con punto flotante sin conversión explícita:

```
fn main() {
    let entero_pequeno: u8 = 200;

    // Casting seguro hacia un tipo más grande (widening cast)
    let entero_grande: u64 = entero_pequeno as u64;

    // Truncamiento en casting hacia un tipo más pequeño (narrowing
cast)
    let valor_original: u16 = 300;
    let truncado: u8 = valor_original as u8; // 300 % 256 = 44

    let factor: f64 = 3.75;
    let parte_entera: i32 = factor as i32; // Trunca decimales: 3

    println!("Grande: {}, Truncado: {}, Entero: {}", entero_grande,
truncado, parte_entera);
}
```

1.4 Shadowing (no es lo mismo que mut)

```
fn main() {
    let x = 5;
    let x = x + 1;    // "sombrea" la x anterior: es una variable
NUEVA
    let x = x * 2;
    println!("x final: {}", x); // 12

    // Shadowing permite cambiar de TIPO, mut no lo permite:
    let espacios = "   ";           // &str
    let espacios = espacios.len(); // ahora usize – tipo distinto,
mismo nombre
    println!("espacios: {}", espacios);
}
```

Diferencia clave con mut: mut permite cambiar el *valor* manteniendo el *tipo*. Shadowing crea una variable *nueva* que puede tener un *tipo distinto*, y la anterior queda inaccesible pero sigue existiendo hasta que termine su scope (importante cuando veamos Drop en Ownership).

1.5 Tipos Primitivos

```
fn main() {
    // Enteros: i8/16/32/64/128, u8/16/32/64/128, isize/usize
    let a: i32 = -42;           // por defecto i32 si no se especifica
    let b: u8 = 255;           // 0 a 255
    let indice: usize = 10;    // usize: tamaño de puntero, se usa
-                               para indexar colecciones

    // Punto flotante: f32, f64 (f64 es el default)
    let pi: f64 = 3.14159;

    // Booleano
    let activo: bool = true;

    // Carácter: 4 bytes, Unicode Scalar Value (NO como en C)
    let corazon: char = '♥';

    // Tuplas: agrupan tipos distintos, tamaño fijo
    let punto: (f64, f64, i32) = (3.0, 4.0, 1);
    let (px, py, _pz) = punto; // destructuring
    println!("{}", px, py);

    // Arrays: mismo tipo, tamaño fijo conocido en compile-time
    let dias: [&str; 3] = ["lunes", "martes", "miércoles"];
    println!("{}", dias[0]);
    println!("{}", px, py, indice, pi, activo);
    println!("{}", corazon);
}
```

⚠ **Nota sobre overflow:** en modo debug, sumar `255u8 + 1` produce **panic** (crash controlado). En modo release, hace *wrapping* silencioso (vuelve a 0). Esto es una trampa común para quien viene de C/C++. Usa `checked_add`, `wrapping_add`, o `saturating_add` cuando el comportamiento importe.

1.6 Control de Flujo

```

fn main() {
    // if es una EXPRESIÓN, no una sentencia – devuelve un valor
    let numero = 7;
    let paridad = if numero % 2 == 0 { "par" } else { "impar" };
    println!("{}", numero, paridad);

    // loop: bucle infinito que puede DEVOLVER un valor con break
    let mut contador = 0;
    let resultado = loop {
        contador += 1;
        if contador == 10 {
            break contador * 2; // loop como expresión
        }
    };
    println!("resultado del loop: {}", resultado);

    // while: condicional clásico
    let mut n = 3;
    while n != 0 {
        println!("cuenta regresiva: {}", n);
        n -= 1;
    }

    // for: iteración idiomática sobre rangos y colecciones
    for i in 0..5 { // rango exclusivo [0,5)
        print!("{}", i);
    }
    println!();

    for i in (0..=5).rev() { // rango inclusivo, invertido
        print!("{}", i);
    }
    println!();
}

```

⚠ Error típico del compilador #2: tipos incompatibles en if

```
fn main() {
    let x = 5;
    let y = if x > 0 { "positivo" } else { -1 }; // ❌
}
```

```
error[E0308]: `if` and `else` have incompatible types
  |
  |         let y = if x > 0 { "positivo" } else { -1 };
  |         -----                ^^ expected `&str`,
found integer
  |
  |         expected because of this
```


Por qué importa: como if es una expresión con un tipo, **ambas ramas deben devolver el mismo tipo**. Esto obliga a que tu lógica sea coherente desde el diseño, no en runtime. Es el primer sabor de cómo el sistema de tipos de Rust previene bugs antes de ejecutar una sola línea.

1.7 Funciones

```
fn cuadrado(x: i32) -> i32 {
    x * x // sin ; → es el valor de retorno (expresión, no
sentencia)
}

fn saludar(nombre: &str) {
    println!("Hola, {}!", nombre); // sin retorno explícito →
devuelve () (unit)
}

fn main() {
    println!("{}", cuadrado(5));
    saludar("Rust");
}
```

🔑 **Regla de oro:** si la última línea de una función NO tiene `;`, es el valor de retorno. Si le pones `;` por error, se convierte en sentencia y la función devuelve `()`, causando un error de tipos.

⚠ Error típico del compilador #3

```
fn cuadrado(x: i32) -> i32 {
    x * x; // ❌ el ; convierte esto en sentencia
}
```

```
error[E0308]: mismatched types
--> src/main.rs:1:33
   |
1  | fn cuadrado(x: i32) -> i32 {
   |     ^^^^^^^^^          ^^^ expected `i32`, found `()`
2  |     x * x;
   |     - help: consider removing this semicolon
```

Fíjate que **de nuevo** el compilador te da el help: exacto. Este patrón se repetirá durante todo el curso: aprender Rust es, en gran parte, aprender a leer bien estos mensajes.

Ejercicio Integrador — Módulo 1

Proyecto: Conversor de Temperaturas con Menú

Crea un proyecto nuevo (cargo new conversor_temp) que:

4. Defina una función `celsius_a_fahrenheit(c: f64) -> f64` y `fahrenheit_a_celsius(f: f64) -> f64`.
5. Use un loop que simule un menú (usa valores hardcoded por ahora, la entrada de teclado real la veremos con `std::io` si hace falta, o puedes ya intentarlo con `std::io::stdin`).
6. Use shadowing para transformar un String leído por teclado en un f64 parseado.
7. Use if/else como expresión para mostrar si la temperatura resultante es "congelante", "templada" o "calurosa".
8. Use un array de 3 temperaturas de prueba y un for para mostrar la conversión de cada una.

Pista de arranque para la parte de lectura de teclado (usa expect por ahora, entenderás por qué es "mala práctica" cuando lleguemos a Result en el Módulo 4):

```
use std::io;
```

```
fn main() {  
    let mut entrada = String::new();  
    io::stdin().read_line(&mut entrada).expect("Error al leer");  
    let numero: f64 = entrada.trim().parse().expect("No es un número válido");  
    println!("Leíste: {}", numero);  
}
```